Assignment 6: REST API Project

Overview

Design and build a RESTful API using Node.js and Express for a resource of your choosing (e.g. notes, tasks, products, recipes), backed by a real database. This assignment is the back-end foundation many students extend into their capstone project.

Learning Objectives

- Design and implement RESTful routes following standard conventions
- Perform full CRUD operations against a persistent database
- Return appropriate HTTP status codes and error responses
- Document and test an API independently of a front end

Requirements

- An Express server exposing a single primary resource with full CRUD: list all, get one, create, update, delete
- Routes follow RESTful naming conventions (e.g. GET /items, GET /items/:id, POST /items, PUT /items/:id, DELETE /items/:id)
- Correct HTTP status codes are returned (200, 201, 400, 404, 500 as appropriate)
- The API is connected to a real database (MongoDB, PostgreSQL, or SQLite) — no in-memory-only data
- Input validation on create/update routes, with meaningful JSON error messages on invalid input
- A README documenting how to run the server and listing every available route with example request/response bodies

Technical Specifications

- Environment variables (e.g. database connection string) must be stored in a .env file and excluded via .gitignore
- Include a Postman or Thunder Client collection (or clear curl examples) demonstrating each route works
- Server should not crash on malformed input or a missing record — errors should be caught and handled

Submission Guidelines

- Submit a GitHub repository link and a zipped project folder
- Include your Postman/Thunder Client collection export or equivalent test evidence in the repo and zipped file

Grading Rubric

Criterion	Points	Description
CRUD completeness & correctness	30	All five core operations work as expected against real data
RESTful design & status codes	20	Routes follow REST conventions; status codes match outcomes
Database integration	20	Data is correctly persisted, retrieved, and updated in the database
Error handling & validation	15	Invalid input and missing records are handled gracefully with clear messages
Documentation	15	README clearly documents setup and every route with examples
Total	**100**	

Note: Submissions that do not run, fail to load, or throw uncaught errors on open may receive a substantial deduction across all criteria, since graders cannot evaluate functionality that isn't demonstrable.